SPECS App Store

Full Project Documentation

CodeCanvas campus community app store | React + Supabase on Vercel

1. Overview

CodeCanvas has two faces: a **public app** (this repo) used by every student, and a **private admin console** (separate repo, developers only) for moderation, user inspection, and audit. Both talk to one shared Supabase project — the app through Row Level Security and audited RPCs, the admin through an allowlisted server layer.

2. Technology stack

Layer	Choice
UI	React 18 + TypeScript (strict) + Vite 8
Styling	Tailwind CSS v4, toon visual language, 9 website palettes + dark Midnight
Icons	Phosphor Icons, Lucide (admin)
Markdown	react-markdown + remark-gfm (raw HTML ignored, links allowlisted)
Backend	Supabase (Postgres + Auth + PostgREST RPCs), anon key only, RLS everywhere
Auth	Email/password (bcrypt, Supabase), Google OAuth (auth-code), Turnstile CAPTCHA
Hosting	Vercel (production), Supabase Cloud (data)
Tests	Vitest, colocated src/**/*.test.ts

3. Repository layout

```
src/
  pages/        Main (router), Store (app shell + 7 views), Login,
Landing, About, routes
  components/   Cards, dialogs, bell, voters, markdown, avatar,
dropdown, boundaries, ...
  data/         Repo contracts + local / supabase / http implementations
  utils/        Notifications, rank, palette, time, images, sounds,
urls, vote queue
  auth/         Session provider + Google account types
supabase/       schema.sql (fresh installs) + migration_*.sql (existing
DBs, re-runnable)
soundEffects/   Bundled vote/notification chimes (mp3)
```

4. Architecture (non-negotiable rules)

- UI code talks **only** to AuthRepo / AppRepo / ProfileRepo (src/data/repositories.ts) via src/data/factory.ts. Components never touch localStorage, fetch, or Supabase clients directly.
- One contract, three backends switched by VITE_DATA_SOURCE: local (browser demo, never shipped), supabase (production), http (future custom REST backend, contract in BACKEND_READINESS.md).
- A repository method added in one backend must be implemented in all three.

5. Features

- **Discover** — search/filter student projects; freshness upload stamps.
- **Submit a project** — screenshots (canvas-compressed), docs (write/upload/GitHub import), collaborators, categories. Publishing awards **+20 developer-board points** with a celebration popup.

- **Feedback** — nested reply threads with likes; authors delete their own rows.
- **Notifications inbox** — bell for comments, replies, uplikes, comment likes, collaborator tags, new projects, feed updates; newest-arrival-first, sounds, mark-read, delete-all. Vote/like events are per-user appearances: unvote-then-revote notifies again.
- **Voters viewer** — exactly who upvoted, with profile links.
- **Collaborators** — picker or @mentions; tagged developers get notified.
- **Profiles** — name/role/bio/photo/avatar color, per-page theme, duplicate names impossible (case-insensitive unique index).
- **Community feed** — milestone wall with cheers and nested discussion.
- **Leaderboards** — developers (upvotes + 20 per project) and projects (votes only), top-3 trophies.
- **Settings** — website palette (9 + Midnight dark), sound mute with preview.
- **Accounts** — email/password signup with duplicate detection, Google sign-in, password recovery by email, optional password for Google-only accounts (with a one-time snoozable reminder), Turnstile human check.

6. Database

Tables: profiles, apps, app_votes, comments (nested via parent_id), comment_likes, milestones, milestone_cheers, plus admin-side admins, admin_audit, rate_hits, vote_timeouts. Conventions:

- Public read everywhere; owners manage their own rows; own-row deletes.
- Counters mutate **only** through SECURITY DEFINER RPCs (toggle_app_vote, toggle_comment_like, toggle_milestone_cheer, delete_project, claim_profile [retired], check_rate, check_vote_spam).
- Inserts bind authorship (auth.uid()), votes are trigger-guarded, display names are unique (lower(btrim(name))), URLs and image URLs are CHECKed.
- Anti-spam: vote burst (5/10s) or sustained chains (10 × 2–5s gaps) pause voting 5 minutes; comments 10/min, milestones 5/min.
- One supabase/migration_*.sql per change (drop-if-exists, re-runnable) + the same change mirrored in schema.sql for fresh installs.

7. Authentication flows

- **Email/password:** signup (human check, duplicate detection, confirmation email when enabled) → signin (generic error, no enumeration) → session.
- **Google:** auth-code redirect to /discover (Supabase verifies with the provider; unverified emails never hijack accounts).
- **Recovery:** “Forgot password?” → reset email → /signin?mode=recovery exchanges the code → set-password form. Google-only accounts gain a password this way; Settings also offers add/change password anytime.
- Sessions are Supabase JWTs; sign-out always clears locally even if the server call fails; the human-check widget refreshes every attempt (single-use tokens).

8. Security model

- Authorization lives in RLS/RPCs, never only in hidden buttons.
- XSS: no dangerouslySetInnerHTML, no raw-HTML markdown, protocol allowlists on every rendered link/image (src/utils/urls.ts), canvas re-encoded uploads, strict CSP + HSTS via vercel.json.
- Secrets: .env gitignored and never committed; only anon key + client IDs (public by design) reach the browser; service-role keys never enter the repo.
- Abuse: Turnstile on auth, DB throttles on writes, BFF rate limits, deny telemetry (log_admin_deny) feeding admin_audit (append-only).

9. Environment variables

Variable	Where	Purpose
VITE_GOOGLE_CLIENT_ID	Browser	Google sign-in button
VITE_SUPABASE_URL	Browser	Supabase project endpoint

Variable	Where	Purpose
VITE_SUPABASE_ANON_KEY	Browser	Public anon key (RLS protects)
VITE_DATA_SOURCE	Build	local | supabase | http (production: supabase)
VITE_TURNSTILE_SITE_KEY	Browser	Human-check widget (public site key)
VITE_API_URL	Browser	Only for http custom-backend mode

10. Commands

```
npm install
npm run dev      # http://localhost:3000
npm run build    # production build (dist/ is gitignored)
npm run preview  # serve the production build
npm test         # vitest suite (11 files / 50 tests)
npm run format   # prettier-write
```

Typecheck: npx tsc --noEmit (must pass). Audit deps: npm audit.

11. Deployment

- Push to main → Vercel builds and deploys. Commit authors must match a GitHub identity with project access or the deploy is blocked.
- Supabase Auth → URL Configuration must allowlist the prod origin plus /discover and /signin?mode=recovery; Google Cloud OAuth needs the prod origin in authorized origins and the Supabase callback in redirect URIs.
- Attack Protection: Turnstile provider + secret, leaked-password blocking.
- Rollback: Vercel Deployments → Redeploy a known-good build (DB changes go forward only — see RUNBOOK.md for backup/restore and the incident table).

12. Testing

Vitest covers URL sanitizers, notifications engine (ordering, re-votes, re-likes, re-stamps), palettes, timestamps, ranking bonus, auth duplicates, signup honesty, profile name guards, and password recovery. Admin console has no runner yet. Critical flows without coverage: vote-toggle rollback, RLS/IDOR integration, admin RPC allowlists.

13. Troubleshooting

Symptom	Likely cause
Google login loops to localhost	Supabase redirect allowlist still points at localhost
“Provider is not enabled”	Google provider off in Supabase Auth settings
“Origin not allowed” (Google button)	Prod origin missing in Cloud Console origins
Login 400s after enabling CAPTCHA	App bundle predates the Turnstile wiring — redeploy
Votes/comments never save	RLS migration not run in the SQL editor
Theme reverts after picking	Duplicate display name blocked the save (resolved: names unique)
Bell silent on re-votes	Old bundle — per-voter notifications need the latest deploy

14. Team & license

Built by the SPECS student developer community (see the About page in-app for the roster).
Internal campus project — contact parsu.specs@gmail.com.